```cpp
// share_server.cpp (for Arch Linux)
#include <iostream>
#include <filesystem>
#include <fstream>
#include <regex>
#include <vector>
#include <map>
#include <cstring>
#include <algorithm>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <sys/stat.h>
#include <httplib.h>
#include <nlohmann/json.hpp>

using json = nlohmann::json;
namespace fs = std::filesystem;

const int PORT = 8000;
std::string SHARE_FOLDER;

// Get local IP address
std::string get_local_ip() {
    try {
        int sock = socket(AF_INET, SOCK_DGRAM, 0);
        if (sock == -1) return "127.0.0.1";

        struct sockaddr_in servaddr;
        servaddr.sin_addr.s_addr = inet_addr("8.8.8.8");
        servaddr.sin_family = AF_INET;
        servaddr.sin_port = htons(80);

        if (connect(sock, (struct sockaddr*)&servaddr, sizeof(servaddr)) < 0) {
            close(sock);
            return "127.0.0.1";
        }

        struct sockaddr_in name;
        socklen_t namelen = sizeof(name);
        getsockname(sock, (struct sockaddr*)&name, &namelen);
        close(sock);

        return inet_ntoa(name.sin_addr);
    } catch (...) {
        return "127.0.0.1";
    }
}

// URL decode function
std::string url_decode(const std::string& src) {
    std::string result;
    for (size_t i = 0; i < src.length(); ++i) {
        if (src[i] == '%' && i + 2 < src.length()) {
            int hex_value;
            std::sscanf(src.substr(i + 1, 2).c_str(), "%x", &hex_value);
            result += static_cast<char>(hex_value);
            i += 2;
        } else if (src[i] == '+') {
            result += ' ';
```

```
        } else {
            result += src[i];
        }
    }
    return result;
}

// Format file size (bytes to human readable)
std::string format_file_size(size_t bytes) {
    if (bytes == 0) return "-";

    const char* units[] = {"B", "KB", "MB", "GB"};
    double size = static_cast<double>(bytes);
    int unit_index = 0;

    while (size ⩾ 1024 && unit_index < 3) {
        size /= 1024;
        unit_index++;
    }

    char buffer[32];
    std::snprintf(buffer, sizeof(buffer), "%.2f %s", size, units[unit_index]);
    return std::string(buffer);
}

// Check if path is safe (no directory traversal)
bool is_path_safe(const std::string& path) {
    try {
        std::string abs_path = fs::absolute(path).string();
        std::string abs_share = fs::absolute(SHARE_FOLDER).string();
        return abs_path.compare(0, abs_share.length(), abs_share) == 0;
    } catch (...) {
        return false;
    }
}

// API endpoint: List files
void handle_list_api(const httplib::Request& req, httplib::Response& res) {
    std::string folder_path = url_decode(req.path.substr(9)); // Remove "/api/list"

    if (folder_path.empty() || folder_path == "/") {
        folder_path = SHARE_FOLDER;
    } else {
        folder_path = SHARE_FOLDER + "/" + folder_path;
    }

    if (!is_path_safe(folder_path)) {
        res.set_header("Content-Type", "application/json");
        res.status = 403;
        res.set_content("[]", "application/json");
        return;
    }

    if (!fs::is_directory(folder_path)) {
        res.set_header("Content-Type", "application/json");
        res.status = 404;
        res.set_content("[]", "application/json");
        return;
    }

    json items = json::array();
```

```
    std::vector<std::string> entries;

    try {
        for (const auto& entry : fs::directory_iterator(folder_path)) {
            entries.push_back(entry.path().filename().string());
        }
        std::sort(entries.begin(), entries.end());

        for (const auto& item : entries) {
            std::string item_path = folder_path + "/" + item;
            bool is_dir = fs::is_directory(item_path);
            size_t file_size = is_dir ? 0 : fs::file_size(item_path);

            std::string rel_path = fs::relative(item_path, SHARE_FOLDER).string();
            std::replace(rel_path.begin(), rel_path.end(), '\\', '/');

            json item_obj;
            item_obj["name"] = item;
            item_obj["isDir"] = is_dir;
            item_obj["size"] = file_size;
            item_obj["path"] = "/" + rel_path;

            items.push_back(item_obj);
        }
    } catch (...) {
        // Permission denied
    }

    res.set_header("Content-Type", "application/json");
    res.set_content(items.dump(), "application/json");
}

// API endpoint: Upload files
void handle_upload_api(const httplib::Request& req, httplib::Response& res) {
    std::string upload_path = url_decode(req.path.substr(11)); // Remove "/api/upload"

    std::string target_folder;
    if (upload_path.empty() || upload_path == "/") {
        target_folder = SHARE_FOLDER;
    } else {
        target_folder = SHARE_FOLDER + "/" + upload_path;
    }

    if (!is_path_safe(target_folder)) {
        res.status = 403;
        res.set_content(R"({"success":false,"error":"Access denied"})",
"application/json");
        return;
    }

    if (!fs::is_directory(target_folder)) {
        res.status = 404;
        res.set_content(R"({"success":false,"error":"Folder not found"})",
"application/json");
        return;
    }

    try {
        json uploaded_files = json::array();

        // Parse multipart form data
```

```
        std::string content_type = req.get_header_value("Content-Type");
        std::regex boundary_regex(R"(boundary=([^\s;]+))");
        std::smatch match;

        if (!std::regex_search(content_type, match, boundary_regex)) {
            res.status = 400;
            res.set_content(R"({"success":false,"error":"No boundary found"})",
"application/json");
            return;
        }

        std::string boundary = "--" + std::string(match[1]);
        std::string body = req.body;
        size_t pos = 0;

        while ((pos = body.find(boundary, pos)) != std::string::npos) {
            pos += boundary.length();

            if (body.substr(pos, 2) == "--") break; // End boundary

            // Skip CRLF
            if (body.substr(pos, 2) == "\r\n") pos += 2;
            else if (body[pos] == '\n') pos += 1;

            // Find headers
            size_t header_end = body.find("\r\n\r\n", pos);
            if (header_end == std::string::npos) {
                header_end = body.find("\n\n", pos);
                if (header_end == std::string::npos) continue;
            }

            std::string headers = body.substr(pos, header_end - pos);

            // Extract filename
            std::regex filename_regex(R"(filename="([^"]+)")");
            if (!std::regex_search(headers, match, filename_regex)) continue;

            std::string filename = match[1];
            filename = fs::path(filename).filename().string();
            if (filename.empty()) continue;

            // Get file content
            size_t content_start = header_end + 4;
            size_t content_end = body.find(boundary, content_start);
            if (content_end == std::string::npos) content_end = body.length();

            // Remove trailing CRLF/LF before boundary
            while (content_end > content_start &&
                   (body[content_end - 1] == '\n' || body[content_end - 1] == '\r')) {
                content_end--;
            }

            std::string file_content = body.substr(content_start, content_end -
content_start);

            // Save file
            std::string file_path = target_folder + "/" + filename;
            std::ofstream file(file_path, std::ios::binary);
            file.write(file_content.c_str(), file_content.length());
            file.close();
```

```
            json file_obj;
            file_obj["name"] = filename;
            file_obj["size"] = file_content.length();
            uploaded_files.push_back(file_obj);
        }

        json response;
        response["success"] = true;
        response["files"] = uploaded_files;

        res.set_header("Content-Type", "application/json");
        res.set_content(response.dump(), "application/json");

    } catch (const std::exception& e) {
        json error_response;
        error_response["success"] = false;
        error_response["error"] = e.what();

        res.status = 500;
        res.set_header("Content-Type", "application/json");
        res.set_content(error_response.dump(), "application/json");
    }
}

// Get MIME type
std::string get_mime_type(const std::string& filename) {
    static const std::map<std::string, std::string> mime_types = {
        {".html", "text/html"},
        {".css", "text/css"},
        {".js", "application/javascript"},
        {".json", "application/json"},
        {".png", "image/png"},
        {".jpg", "image/jpeg"},
        {".jpeg", "image/jpeg"},
        {".gif", "image/gif"},
        {".svg", "image/svg+xml"},
        {".pdf", "application/pdf"},
        {".txt", "text/plain"},
        {".mp4", "video/mp4"},
        {".mp3", "audio/mpeg"},
    };

    std::string ext = filename.substr(filename.find_last_of("."));
    std::transform(ext.begin(), ext.end(), ext.begin(), ::tolower);

    auto it = mime_types.find(ext);
    return it != mime_types.end() ? it->second : "application/octet-stream";
}

// HTML content (embedded from the original Python)
std::string get_html() {
    return R"(<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>File Share</title>
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body {
            font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
```

```
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    min-height: 100vh;
    display: flex;
    justify-content: center;
    align-items: center;
    padding: 20px;
}
.container {
    background: white;
    border-radius: 12px;
    box-shadow: 0 10px 40px rgba(0, 0, 0, 0.2);
    width: 100%;
    max-width: 900px;
    overflow: hidden;
}
.header {
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    color: white;
    padding: 30px;
    text-align: center;
}
.header h1 { font-size: 28px; margin-bottom: 8px; }
.breadcrumb {
    background: #f5f5f5;
    padding: 12px 20px;
    font-size: 14px;
    color: #666;
    border-bottom: 1px solid #e0e0e0;
    display: flex;
    align-items: center;
    gap: 8px;
    flex-wrap: wrap;
}
.breadcrumb a {
    color: #667eea;
    text-decoration: none;
    cursor: pointer;
    padding: 4px 8px;
    border-radius: 4px;
    transition: background 0.2s;
}
.breadcrumb a:hover { background: #e8e8ff; }
.content { padding: 20px; min-height: 300px; }
.upload-area {
    border: 2px dashed #667eea;
    border-radius: 8px;
    padding: 30px;
    text-align: center;
    cursor: pointer;
    transition: all 0.3s;
    background: #f8f9ff;
}
.upload-area:hover {
    background: #f0f0ff;
    border-color: #764ba2;
}
.upload-area.dragover {
    background: #e8e8ff;
    border-color: #764ba2;
    box-shadow: 0 4px 12px rgba(102, 126, 234, 0.3);
}
```

```
        .file-grid {
            display: grid;
            grid-template-columns: repeat(auto-fill, minmax(150px, 1fr));
            gap: 15px;
            margin-bottom: 20px;
            margin-top: 20px;
        }
        .file-item {
            background: #f9f9f9;
            border: 1px solid #e0e0e0;
            border-radius: 8px;
            padding: 15px;
            text-align: center;
            cursor: pointer;
            transition: all 0.3s;
            text-decoration: none;
            color: #333;
        }
        .file-item:hover {
            background: #f0f0ff;
            border-color: #667eea;
            transform: translateY(-2px);
            box-shadow: 0 4px 12px rgba(102, 126, 234, 0.2);
        }
        .file-icon { font-size: 36px; margin-bottom: 10px; }
        .file-name { font-size: 13px; font-weight: 500; word-break: break-word; }
        .file-size { font-size: 11px; color: #999; margin-top: 8px; }
        .empty-state { text-align: center; padding: 60px 20px; color: #999; }
                .loader { text-align: center; padding: 40px 20px; color: #667eea; }
                .spinner {
                    border: 4px solid #f0f0f0;
                    border-top: 4px solid #667eea;
                    border-radius: 50%;
                    width: 40px;
                    height: 40px;
                    animation: spin 0.8s linear infinite;
                    margin: 0 auto 15px;
                }
                @keyframes spin { 0% { transform: rotate(0deg); } 100% { transform:
rotate(360deg); } }
                .footer { background: #f5f5f5; padding: 15px 20px; text-align: center;
font-size: 12px; color: #999; border-top: 1px solid #e0e0e0; }
                .alert { padding: 12px 16px; border-radius: 6px; margin-bottom: 15px;
font-size: 14px; }
                .alert-success { background: #d4edda; border: 1px solid #c3e6cb; color:
#155724; }
                .alert-error { background: #f8d7da; border: 1px solid #f5c6cb; color:
#721c24; }
                #fileInput { display: none; }
                .upload-progress { display: none; margin-top: 15px; }
                .progress-bar { width: 100%; height: 8px; background: #e0e0e0; border-
radius: 4px; overflow: hidden; }
                .progress-fill { height: 100%; background: linear-gradient(90deg, #667eea
0%, #764ba2 100%); width: 0%; transition: width 0.3s; }
                .upload-status { font-size: 13px; color: #667eea; margin-top: 8px; font-
weight: 500; }
                @media (max-width: 600px) {
                    .file-grid { grid-template-columns: repeat(auto-fill, minmax(120px,
1fr)); gap: 10px; }
                    .header h1 { font-size: 20px; }
                }
```

```
            </style>
        </head>
        <body>
            <div class="container">
                <div class="header">
                    <h1>📁 File Share</h1>
                </div>
                <div class="breadcrumb" id="breadcrumb"></div>
                <div class="content" id="content">
                    <div class="upload-section">
                        <div class="upload-area" id="uploadArea">
                            <div class="upload-icon">📤</div>
                            <div class="upload-text">Drag files here or click to
select</div>
                            <div class="upload-subtext">Multiple files supported</div>
                        </div>
                        <input type="file" id="fileInput" multiple>
                        <div class="upload-progress" id="uploadProgress">
                            <div class="progress-bar">
                                <div class="progress-fill" id="progressFill"></div>
                            </div>
                            <div class="upload-status" id="uploadStatus">Uploading...
</div>
                        </div>
                    </div>
                    <div id="alerts"></div>
                    <div class="loader" id="loader">
                        <div class="spinner"></div>
                        <p>Loading...</p>
                    </div>
                    <div class="file-grid" id="fileGrid"></div>
                </div>
                <div class="footer">Sharing folder contents</div>
            </div>

            <script>
                let currentPath = "/";
                const icons = {
                    folder: "📁",
                    image: "🖼",
                    video: "🎥",
                    audio: "🎵",
                    pdf: "📄",
                    zip: "📦",
                    code: "📜",
                    text: "📝",
                    file: "📄"
                };

                const uploadArea = document.getElementById("uploadArea");
                const fileInput = document.getElementById("fileInput");
                const uploadProgress = document.getElementById("uploadProgress");
                const progressFill = document.getElementById("progressFill");
                const uploadStatus = document.getElementById("uploadStatus");
                const alerts = document.getElementById("alerts");

                uploadArea.addEventListener("click", () => fileInput.click());
                uploadArea.addEventListener("dragover", (e) => {
                    e.preventDefault();
                    uploadArea.classList.add("dragover");
                });
```

```
            uploadArea.addEventListener("dragleave", () => {
                uploadArea.classList.remove("dragover");
            });
            uploadArea.addEventListener("drop", (e) => {
                e.preventDefault();
                uploadArea.classList.remove("dragover");
                handleFiles(e.dataTransfer.files);
            });
            fileInput.addEventListener("change", (e) => {
                handleFiles(e.target.files);
            });

            function handleFiles(files) {
                if (files.length === 0) return;
                const formData = new FormData();
                for (let file of files) {
                    formData.append("files", file);
                }
                uploadProgress.style.display = "block";
                progressFill.style.width = "0%";
                uploadStatus.textContent = `Uploading ${files.length} file(s)...`;

                fetch(`/api/upload${currentPath}`, {
                    method: "POST",
                    body: formData
                })
                .then(res => res.json())
                .then(data => {
                    uploadProgress.style.display = "none";
                    fileInput.value = "";
                    if (data.success) {
                        showAlert(`✓ Successfully uploaded ${data.files.length}
file(s)!`, "success");
                        loadFiles();
                    } else {
                        showAlert(`× Upload failed: ${data.error}`, "error");
                    }
                })
                .catch(err => {
                    uploadProgress.style.display = "none";
                    showAlert(`× Upload error: ${err.message}`, "error");
                });
            }

            function showAlert(message, type) {
                const alertDiv = document.createElement("div");
                alertDiv.className = `alert alert-${type}`;
                alertDiv.textContent = message;
                alerts.appendChild(alertDiv);
                setTimeout(() => alertDiv.remove(), 5000);
            }

            function getFileIcon(name, isDir) {
                if (isDir) return icons.folder;
                const ext = name.split(".").pop().toLowerCase();
                if (["jpg", "jpeg", "png", "gif", "svg", "webp"].includes(ext)) return
icons.image;
                if (["mp4", "avi", "mkv", "mov"].includes(ext)) return icons.video;
                if (["mp3", "wav", "flac", "m4a"].includes(ext)) return icons.audio;
                if (ext === "pdf") return icons.pdf;
                if (["zip", "rar", "7z", "tar"].includes(ext)) return icons.zip;
```

```
                    if (["js", "py", "java", "cpp", "html", "css"].includes(ext)) return
icons.code;
                    if (["txt", "md", "doc"].includes(ext)) return icons.text;
                    return icons.file;
                }

                function formatFileSize(bytes) {
                    if (bytes === 0) return "-";
                    const k = 1024;
                    const sizes = ["B", "KB", "MB", "GB"];
                    const i = Math.floor(Math.log(bytes) / Math.log(k));
                    return Math.round((bytes / Math.pow(k, i)) * 100) / 100 + " " +
sizes[i];
                }

                function updateBreadcrumb() {
                    const parts = currentPath.split("/").filter(p => p);
                    let html = `<a onclick="navigate('/')">Home</a>`;
                    let path = "";
                    for (let part of parts) {
                        path += "/" + part;
                        html += `<span>/</span><a onclick="navigate('${path}')">${part}
</a>`;
                    }
                    document.getElementById("breadcrumb").innerHTML = html;
                }

                function navigate(path) {
                    currentPath = path;
                    loadFiles();
                }

                function loadFiles() {
                    updateBreadcrumb();
                    const loader = document.getElementById("loader");
                    const fileGrid = document.getElementById("fileGrid");
                    loader.style.display = "block";
                    fileGrid.innerHTML = "";

                    fetch(`/api/list${currentPath}`)
                    .then(res => res.json())
                    .then(data => {
                        loader.style.display = "none";
                        if (data.length === 0) {
                            fileGrid.innerHTML = `<div class="empty-state" style="grid-
column: 1/-1;"><div class="empty-state-icon">□</div><p>This folder is empty</p></div>`;
                            return;
                        }
                        for (let item of data) {
                            const icon = getFileIcon(item.name, item.isDir);
                            const size = formatFileSize(item.size);
                            if (item.isDir) {
                                const div = document.createElement("div");
                                div.className = "file-item";
                                div.onclick = () => navigate(item.path);
                                div.innerHTML = `<div class="file-icon">${icon}</div><div
class="file-name">${item.name}</div>`;
                                fileGrid.appendChild(div);
                            } else {
                                const a = document.createElement("a");
                                a.className = "file-item";
```

```
                                a.href = item.path;
                                a.download = true;
                                a.innerHTML = `<div class="file-icon">${icon}</div><div
class="file-name">${item.name}</div><div class="file-size">${size}</div>`;
                                fileGrid.appendChild(a);
                            }
                        }
                    })
                    .catch(err => {
                        loader.style.display = "none";
                        fileGrid.innerHTML = `<div class="empty-state" style="grid-column:
1/-1;"><div class="empty-state-icon">✖</div><p>Error loading files</p></div>`;
                    });
            }
            loadFiles();
        </script>
    </body>
    </html>)";
    }

    int main() {
        // Get script folder
        SHARE_FOLDER = fs::current_path().string();

        // Create HTTP server
        httplib::Server svr;

        // Register API endpoints
        svr.Get("/api/list.*", handle_list_api);
        svr.Post("/api/upload.*", handle_upload_api);

        // Serve index.html for root
        svr.Get("/", [](const httplib::Request& req, httplib::Response& res) {
            res.set_header("Content-Type", "text/html");
            res.set_content(get_html(), "text/html");
        });

        svr.Get("/index.html", [](const httplib::Request& req, httplib::Response& res)
{
            res.set_header("Content-Type", "text/html");
            res.set_content(get_html(), "text/html");
        });

        // Serve files
        svr.Get("/.*", [](const httplib::Request& req, httplib::Response& res) {
            std::string file_path = SHARE_FOLDER + req.path;

            if (!is_path_safe(file_path)) {
                res.status = 403;
                return;
            }

            if (fs::is_regular_file(file_path)) {
                std::ifstream file(file_path, std::ios::binary);
                if (file) {
                    std::string mime = get_mime_type(file_path);
                    res.set_header("Content-Type", mime);
                    res.set_header("Content-Disposition",
                        "attachment; filename=" +
fs::path(file_path).filename().string());
```

```
                    std::string content((std::istreambuf_iterator<char>(file)),
                                        std::istreambuf_iterator<char>());
                    res.set_content(content, mime);
                    file.close();
                } else {
                    res.status = 404;
                }
            } else {
                res.status = 404;
            }
        });

        std::string local_ip = get_local_ip();

        std::cout << "\n" << std::string(50, '=') << std::endl;
        std::cout << " FILE SHARE SERVER STARTED" << std::endl;
        std::cout << std::string(50, '=') << std::endl;
        std::cout << " Sharing folder: " << SHARE_FOLDER << std::endl;
        std::cout << " Local access: http://localhost:" << PORT << std::endl;
        std::cout << " Network access: http://" << local_ip << ":" << PORT <<
std::endl;
        std::cout << std::string(50, '=') << std::endl;
        std::cout << "Press Ctrl+C to stop\n" << std::endl;

        svr.listen("0.0.0.0", PORT);

        return 0;
    }
```